Alturas Reservas: Integración de Firebase Authentication y Cloud Firestore en la Aplicación Móvil con React Native

Etapa 3 – Avance 2: Foro 2

Gustavo Adolfo Navarro Martínez (NM181601)

Técnico en Ingeniería en Computación, Universidad Don Bosco

DPS941: Diseño y Programación de Software Multiplataforma

Ing. Alexander Alberto Sigüenza

8 de noviembre de 2026

Alturas Reservas: Integración de Firebase Authentication y Cloud Firestore en la Aplicación Móvil con React Native

Alturas Reservas es el proyecto de cátedra de la asignatura: un sistema de reservas de mesas para la Cafetería Mirador Las Alturas del Boquerón, con una versión web en Next.js y una aplicación móvil en React Native. En el primer avance de la Etapa 3 se construyeron las pantallas móviles a partir de los mockups de Figma, pero los usuarios, la sesión y las reservas se guardaban solo en el teléfono con AsyncStorage. Eso significaba que cada dispositivo tenía sus propios datos, que las contraseñas quedaban almacenadas en el equipo y que dos clientes no podían ver la misma disponibilidad de mesas.

Este segundo avance resuelve esos problemas al integrar Firebase. La autenticación pasa a Firebase Authentication, con correo y contraseña y con cuenta de Google, y los datos del sistema se guardan en Cloud Firestore asociados al usuario autenticado. El documento presenta primero una investigación sobre Firebase y sobre las formas de usarlo desde React Native; después explica cómo se aplicó en el proyecto (arquitectura, flujo de autenticación, modelo de datos, transacciones y reglas de seguridad), muestra evidencias de funcionamiento y termina con las conclusiones y las referencias.

Firebase

Firebase es una plataforma de desarrollo de aplicaciones de Google que ofrece servicios de *backend* administrados: autenticación, bases de datos, almacenamiento de archivos, funciones en la nube, notificaciones y *hosting*, entre otros (Google, s.f.-c). Se le considera un *Backend as a Service* (BaaS) porque la aplicación se conecta directamente a esos servicios mediante un SDK, sin que el equipo tenga que programar, desplegar ni mantener un servidor propio (Moroney, 2017). Para un proyecto pequeño como el de una cafetería esto es una ventaja importante: el plan gratuito Spark cubre el volumen de uso esperado y la seguridad, la escalabilidad y las copias de la infraestructura quedan a cargo de Google.

Firebase Authentication

Firebase Authentication administra la identidad de los usuarios. Ofrece inicio de sesión con correo y contraseña, con enlaces sin contraseña, con número de teléfono y con proveedores federados como Google, Facebook, Apple o GitHub (Google, s.f.-a). El servicio almacena las contraseñas con *hash* y sal, por lo que la aplicación nunca las conoce, y entrega a cada usuario un identificador único (*uid*) y un token de sesión que se renueva automáticamente. También envía los correos de verificación de cuenta y de restablecimiento de contraseña. El SDK informa los cambios de sesión con el evento `onAuthStateChanged`, que permite a la interfaz reaccionar cuando el usuario entra o sale.

Cloud Firestore

Cloud Firestore es una base de datos NoSQL orientada a documentos. Los datos se guardan en documentos, que son conjuntos de campos parecidos a un objeto JSON, y los documentos se agrupan en colecciones (Google, s.f.-b). No requiere un esquema fijo y permite consultas por campo con índices automáticos. Sus características más útiles para este proyecto son tres:

- **Actualizaciones en tiempo real:** con `onSnapshot` la aplicación se suscribe a un documento o a una consulta y recibe los cambios en cuanto ocurren, sin recargar (Google, s.f.-d).
- **Transacciones:** un grupo de lecturas y escrituras se ejecuta de forma atómica; si otro cliente modifica los mismos documentos a la vez, Firestore repite la transacción con los datos nuevos (Google, s.f.-f).
- **Reglas de seguridad:** un lenguaje declarativo, evaluado en el servidor, decide qué usuario puede leer o escribir cada documento según su *uid* y los datos enviados (Google, s.f.-e).

Firebase en React Native

Existen dos formas principales de usar Firebase en una aplicación de React Native (Expo, s.f.-b). La primera es el **SDK web de Firebase** (paquete `firebase`), escrito completamente en JavaScript; funciona en Expo Go sin compilar código nativo y es el mismo que se usa en la web.

La segunda es la librería **React Native Firebase**, que envuelve los SDK nativos de Android e iOS; da acceso a servicios adicionales como Crashlytics o notificaciones push, pero exige un *development build* y configuración nativa. La Tabla 1 resume la comparación.

Tabla 1

Comparación de opciones para usar Firebase en React Native

Criterio	SDK web de Firebase	React Native Firebase
Compatibilidad con Expo Go	Sí, sin pasos adicionales	No, requiere development build
Servicios	Authentication, Firestore, Storage, Functions	Todos, incluidos Crashlytics, Analytics y Messaging
Persistencia de sesión	Con getReactNativePersistence y AsyncStorage	Nativa
Rendimiento	Bueno para el volumen del proyecto	Mejor en cargas grandes (código nativo)
Código compartido con la web	El mismo SDK de la versión web	API diferente a la web

Nota. Elaboración propia a partir de Expo (s.f.-b) y Google (s.f.-c).

Para Alturas Reservas se eligió el SDK web (versión 12) porque permite seguir probando la aplicación en un teléfono físico con Expo Go, igual que en el avance anterior, y porque los servicios necesarios (Authentication y Firestore) están completos en él. En React Native la sesión se guarda con `initializeAuth` y la persistencia `getReactNativePersistence(AsyncStorage)`, de modo que el usuario sigue conectado al cerrar y reabrir la aplicación. AsyncStorage deja de guardar los datos del sistema y solo conserva el token de sesión.

El inicio de sesión con Google necesita un paso más en el teléfono, porque el SDK web no puede abrir la ventana de Google dentro de una aplicación nativa. La librería `expo-auth-session` abre el navegador del sistema, obtiene el *idToken* de Google y se lo entrega a Firebase con `signInWithCredential` (Expo, s.f.-a). Esta opción requiere un ID de cliente OAuth de Android o iOS y un *development build*, ya que Expo Go no permite registrar el

esquema de redirección propio de la aplicación. En la versión web de la misma aplicación se usa directamente `signInWithPopup`.

Integración en Alturas Reservas

Configuración del proyecto

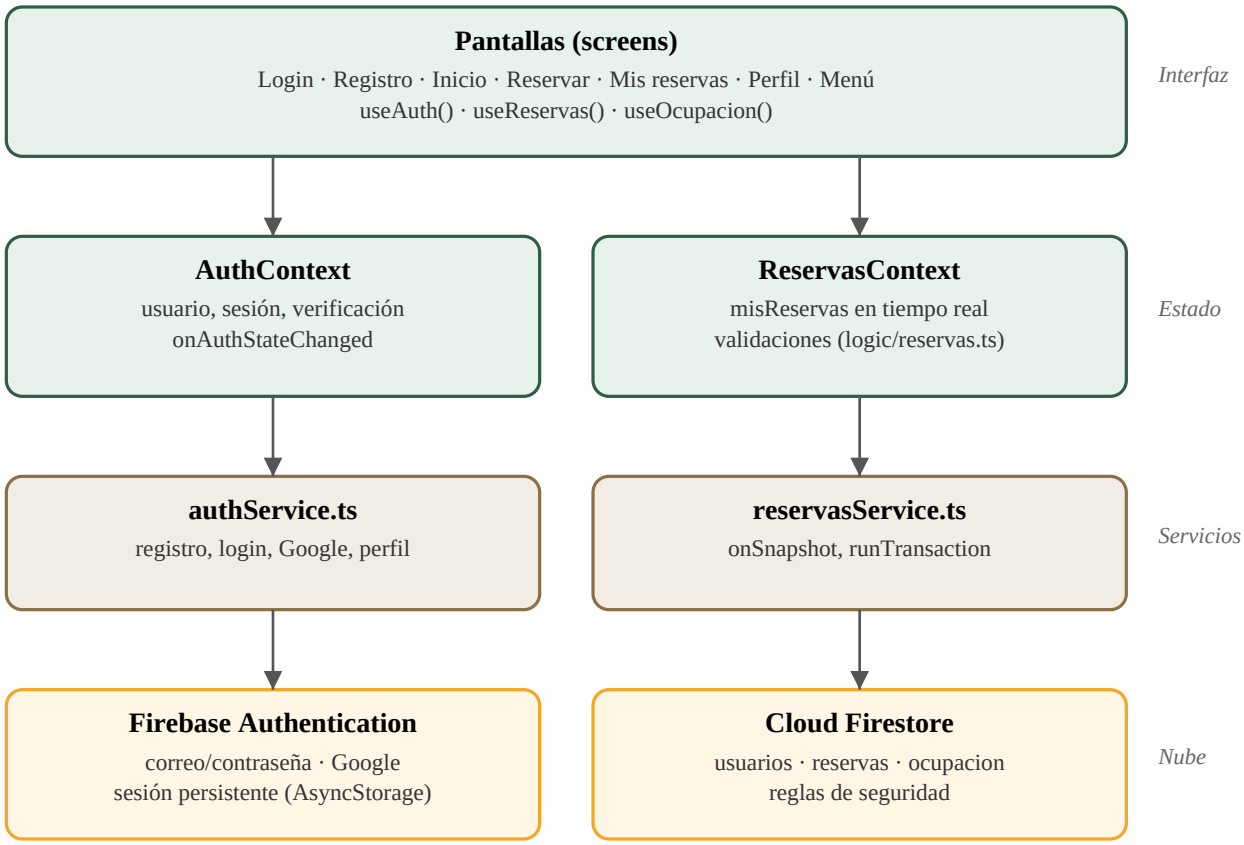
Se creó el proyecto *alturas-reservas* en la consola de Firebase y se registró una aplicación web. En Authentication se habilitaron los proveedores de correo y contraseña y de Google; en Firestore se creó la base de datos en modo producción y se publicaron las reglas del archivo `firestore.rules`. Las claves de la aplicación se guardan en variables `EXPO_PUBLIC_*` de un archivo `.env` que no se sube al repositorio; el archivo `.env.example` y la guía `FIREBASE.md` explican cómo configurarlas. Si faltan las claves, la aplicación muestra un aviso en lugar de fallar. Durante las pruebas, el proyecto se actualizó a Expo SDK 58 para que fuera compatible con la versión vigente de Expo Go en iOS.

Arquitectura

La aplicación mantiene la organización por capas del avance anterior (Figura 1). Las pantallas no llaman a Firebase directamente: usan los contextos de React, y estos usan dos servicios que concentran todo el acceso a la nube. Gracias a esta separación, el cambio de AsyncStorage a Firebase se hizo reemplazando la capa de servicios y los contextos, con cambios pequeños en las pantallas.

Figura 1

Arquitectura por capas de la aplicación móvil con Firebase



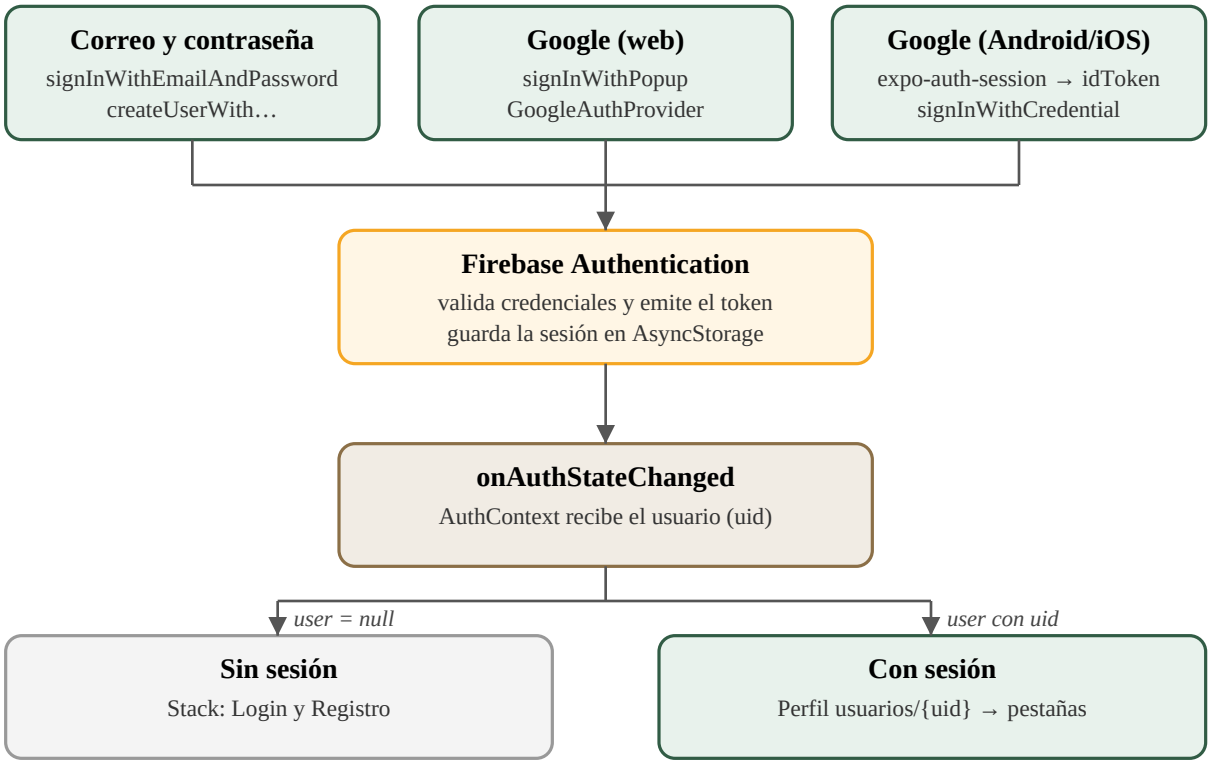
Nota. Las flechas indican dependencias: cada capa solo usa la inmediata inferior. Elaboración propia.

Autenticación

La Figura 2 muestra el flujo de autenticación. Los tres métodos de inicio de sesión terminan en Firebase Authentication; el contexto `AuthContext` escucha `onAuthStateChanged` y, cuando hay un usuario, se suscribe a su documento `usuarios/{uid}`. La navegación decide qué mostrar según ese estado: sin sesión, el *stack* de inicio de sesión y registro; con sesión, la barra de pestañas. Por eso, después de registrarse o de entrar con Google, la aplicación cambia de pantalla sin código de navegación adicional.

Figura 2

Flujo de autenticación con Firebase



Nota. Elaboración propia.

Al registrarse con correo se crea la cuenta en Firebase, se guarda el nombre como *displayName*, se crea el perfil en Firestore con el rol *cliente* y se envía un correo de verificación. Al entrar por primera vez con Google, el perfil se crea con el nombre de la cuenta de Google y la pantalla de perfil pide completar el teléfono. La opción "¿Olvidaste tu contraseña?", que en el avance anterior solo mostraba un aviso, ahora envía el correo de restablecimiento con `sendPasswordResetEmail`.

Las validaciones de formato del Foro 1 se conservan (nombre, correo, teléfono de ocho dígitos y contraseña con letras y números). Los errores que devuelve Firebase se traducen a mensajes en español: por ejemplo, `auth/invalid-credential` se muestra como "Correo o contraseña incorrectos", sin revelar si el correo existe, y `auth/network-request-failed` como "Sin conexión a internet".

Modelo de datos en Firestore

Los datos se organizan en tres colecciones (Tabla 2). Cada perfil usa como identificador el *uid* de Authentication y cada reserva guarda el *uid* de quien la creó, lo que permite consultar las reservas del usuario y proteger el acceso con las reglas.

Tabla 2

Colecciones de Cloud Firestore

Colección	Identificador	Campos	Quién la lee
usuarios	uid del usuario	nombre, correo, telefono, rol, proveedor, creadoEn	El propio usuario (y el administrador)
reservas	Automático	uid, correo, nombre, codigo, fecha, hora, personas, tipo, mesa, estado, creadoEn, canceladoEn	El dueño de la reserva (y el administrador)
ocupacion	fecha_hora_mesa (ej. 2026-11-14_1200_Mesa02)	fecha, hora, mesa, uid, reservaId	Cualquier usuario autenticado

Nota. La colección ocupacion no contiene nombres ni correos; solo indica qué mesas están tomadas.

La colección *ocupacion* resolvió un problema de diseño. Para saber si hay mesa disponible, la aplicación necesita conocer las reservas de todos los clientes en una fecha y hora, pero las reglas no deben permitir que un cliente lea las reservas de otro. La solución fue guardar, junto con cada reserva, una marca de ocupación sin datos personales cuyo identificador combina la fecha, la hora y la mesa. La pantalla de reservar se suscribe a las ocupaciones del día elegido y deshabilita en tiempo real las horas sin mesa para el tamaño del grupo.

Creación de reservas con transacciones

Si dos clientes reservan la misma hora casi al mismo tiempo, ambos podrían ver la mesa libre. Para evitarlo, la reserva se crea dentro de una transacción (Figura 3): se leen las marcas de ocupación de las mesas candidatas, de la más pequeña a la más grande; se elige la primera libre y se escriben la reserva y su ocupación en una sola operación. Como el identificador de la ocupación es único, si otro cliente toma la mesa antes, Firestore repite la transacción y se asigna

la siguiente mesa disponible o se informa que no hay cupo.

Figura 3

Fragmento de reservasService.ts: creación de la reserva en una transacción

```
return runTransaction(db, async tx => {
  let mesaLibre: string | null = null;
  for (const m of candidatas) {          // de menor a mayor capacidad
    const snap = await tx.get(doc(db, 'ocupacion', claveOcupacion(fecha, hora, m.id)));
    if (!snap.exists()) { mesaLibre = m.id; break; }
  }
  if (!mesaLibre) throw new SinMesaError();

  tx.set(refReserva, { uid: usuario.uid, fecha, hora, personas, tipo, mesa: mesaLibre,
                       estado: 'Confirmada', creadoEn: serverTimestamp(), ... });
  tx.set(doc(db, 'ocupacion', claveOcupacion(fecha, hora, mesaLibre)),
         { fecha, hora, mesa: mesaLibre, uid: usuario.uid, reservaId: refReserva.id });
  return { id: refReserva.id, mesa: mesaLibre };
});
```

Nota. Se omitieron algunos campos. La cancelación usa otra transacción que cambia el estado a “Cancelada” y borra la marca de ocupación para liberar la mesa.

Las reglas de negocio del avance anterior se mantienen en la capa de lógica: reservar con al menos una hora de anticipación y hasta 30 días antes, una sola reserva confirmada por día y cancelación hasta dos horas antes. Las fechas de creación y cancelación se guardan con `serverTimestamp()`, la hora del servidor, para que no dependan del reloj del teléfono.

Reglas de seguridad

Las reglas de Firestore son la última línea de defensa: aunque alguien modifique la aplicación, el servidor rechaza las operaciones no permitidas (Google, s.f.-e). La Tabla 3 resume las reglas publicadas.

Tabla 3

Resumen de las reglas de seguridad de Firestore

Colección	Lectura	Escritura permitida
usuarios	Solo el dueño o un administrador	Crear el propio perfil con rol “cliente” y campos fijos; después solo cambiar nombre y teléfono. No se puede borrar.
reservas	Solo el dueño o un administrador	Crear con el uid propio, estado “Confirmada”, 1 a 6 personas, tipo válido y fecha/hora con formato correcto. Actualizar únicamente de “Confirmada” a “Cancelada”. Borrar solo el administrador.
ocupacion	Cualquier usuario autenticado	Crear solo junto con una reserva propia de la misma mesa (se verifica con getAfter). Borrar solo el dueño o un administrador.

Nota. El rol de administrador solo puede asignarse desde la consola de Firebase, nunca desde la aplicación.

Módulos secundarios e historial

Además de la autenticación y las reservas, se completaron los módulos secundarios con datos de Firestore:

- **Perfil:** muestra y permite editar el nombre y el teléfono, guardados en el documento del usuario; indica el método de acceso (correo o Google), si el correo está verificado (con opción de reenviar la verificación) y un resumen de reservas próximas, completadas y canceladas.
- **Mis reservas e historial:** las reservas se leen en tiempo real; las próximas permiten cancelar y el historial se filtra por Todas, Completadas o Canceladas. Cada tarjeta muestra el código de la reserva, la mesa y las fechas de registro y cancelación guardadas en Firestore.
- **Inicio y menú:** la pantalla de inicio muestra la próxima reserva del usuario y el menú conserva las categorías del avance anterior.

Evidencias de Funcionamiento

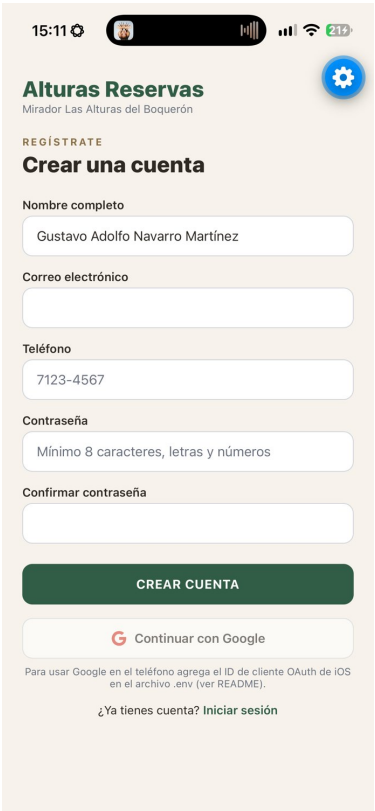
Las figuras 4 a 6 muestran la aplicación ejecutándose en un iPhone físico con Expo Go, conectada al proyecto de Firebase.

Figura 4

Inicio de sesión y registro con Firebase Authentication



Iniciar sesión



Crear una cuenta

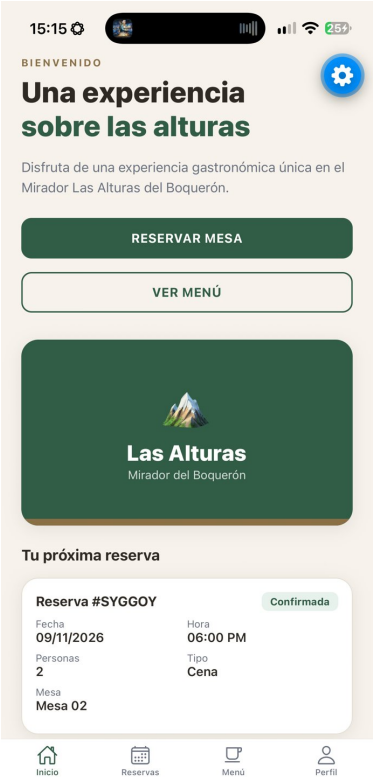
Nota. Ambas pantallas incluyen el botón “Continuar con Google”. En Expo Go aparece desactivado con una indicación, porque en el teléfono requiere un development build; en la versión web funciona con signInWithPopup.

Figura 5

Validación del registro e inicio con la próxima reserva



Validación de la contraseña



Inicio con la próxima reserva

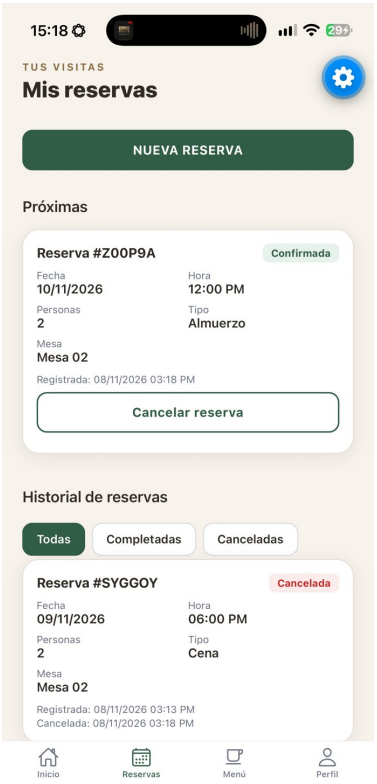
Nota. Las validaciones de formato se ejecutan antes de enviar los datos a Firebase. La tarjeta “Tu próxima reserva” se obtiene de Firestore en tiempo real.

Figura 6

Mis reservas e historial antes y después de cancelar



Reserva confirmada



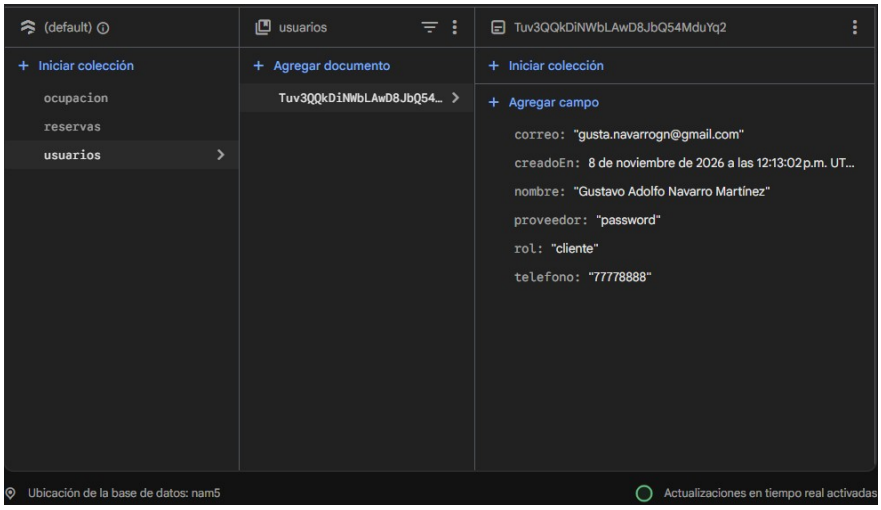
Después de cancelar y crear otra

Nota. Al cancelar, la reserva #SYGGOY pasa al historial con estado “Cancelada” y las fechas de registro y cancelación guardadas con la hora del servidor; la Mesa 02 queda libre y se asigna a la nueva reserva #Z00P9A.

Las figuras 7 a 9 muestran los mismos datos almacenados en Cloud Firestore, vistos desde la consola de Firebase.

Figura 7

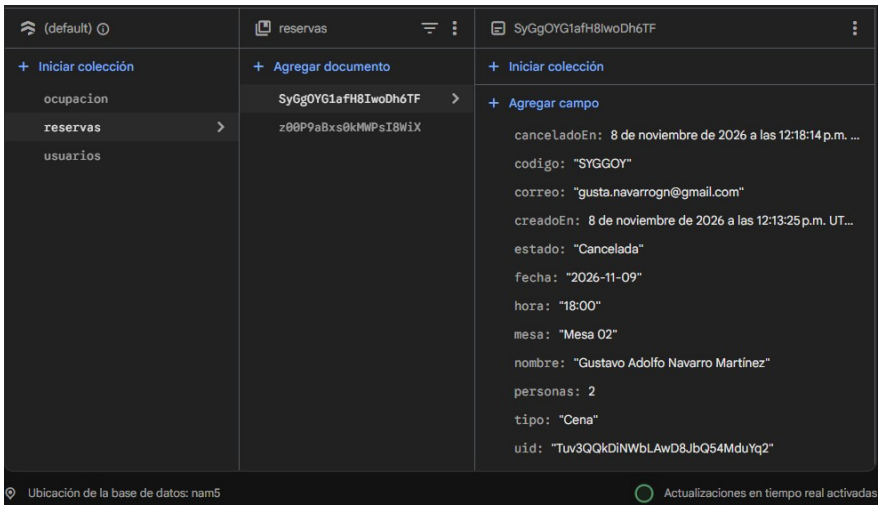
Documento del usuario en la colección usuarios



Nota. El identificador del documento es el uid que asigna Firebase Authentication. El rol se guardó como “cliente” y el proveedor como “password” porque la cuenta se creó con correo y contraseña.

Figura 8

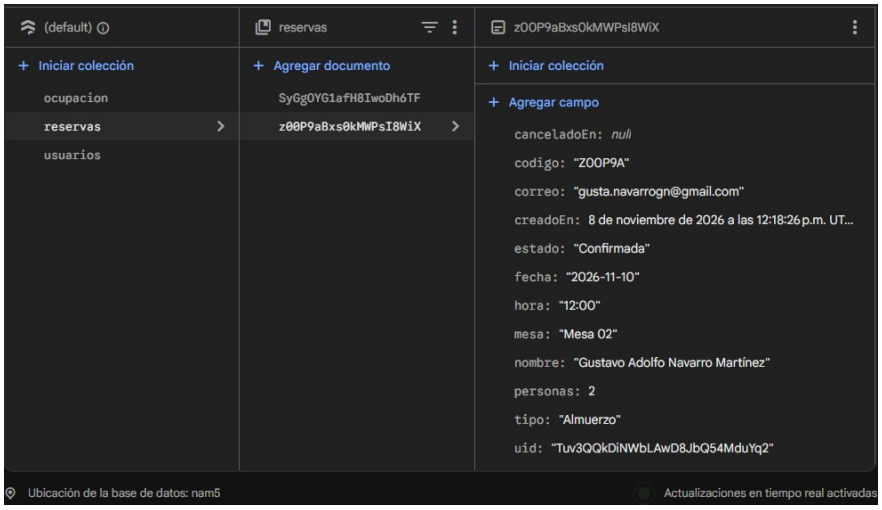
Reserva cancelada en la colección reservas



Nota. La reserva SYGGOY conserva el uid de su dueño y registra creadoEn y canceladoEn con la hora del servidor.

Figura 9

Reserva confirmada en la colección reservas



Nota. La nueva reserva Z00P9A recibió la Mesa 02, liberada por la cancelación anterior; canceladoEn permanece en null. La consola indica “Actualizaciones en tiempo real activadas”.

Recursos del Avance

El funcionamiento de la aplicación en un teléfono físico y en la versión web, la configuración de Firebase y la explicación del código se presentan en el video del avance. Los enlaces son los siguientes:

- Video del avance: https://1drv.ms/v/c/5b529b9580d641c0/IQBG7hV7OoLDS4we70xm_nFIAdRAPjEyjkCzOECGYf_R80E?e=77PBMI
- Repositorio: https://github.com/gustanavarrogn/alturas-reservas (carpeta movil)
- Guía de configuración: movil/FIREBASE.md; reglas de seguridad: movil/firestore.rules

Conclusiones

Firebase permitió convertir una aplicación que guardaba todo en el teléfono en un sistema real de varios usuarios sin construir ni mantener un servidor. Firebase Authentication resolvió de forma segura el registro, el inicio de sesión con correo y con Google, la verificación del correo y la recuperación de la contraseña, tareas que habría sido riesgoso implementar a mano.

Cloud Firestore aportó persistencia en la nube y sincronización en tiempo real, pero su modelo de seguridad obligó a pensar el diseño de los datos: la colección de ocupación y las transacciones surgieron de la necesidad de consultar disponibilidad sin exponer las reservas de otros clientes y sin permitir dos reservas en la misma mesa. Las reglas de seguridad trasladaron al servidor validaciones que antes solo existían en la aplicación.

Elegir el SDK web de Firebase facilitó las pruebas con Expo Go, con la limitación de que el inicio de sesión con Google en el teléfono requiere un *development build*. Por último, la separación en capas definida en el avance anterior permitió reemplazar AsyncStorage por Firebase con cambios mínimos en las pantallas, lo que deja la aplicación preparada para la entrega final.

Referencias

Expo. (s.f.-a). *AuthSession*. Recuperado el 8 de noviembre de 2026, de https://docs.expo.dev/versions/latest/sdk/auth-session/

Expo. (s.f.-b). *Using Firebase*. Recuperado el 8 de noviembre de 2026, de https://docs.expo.dev/guides/using-firebase/

Google. (s.f.-a). *Firebase Authentication*. Firebase. Recuperado el 8 de noviembre de 2026, de https://firebase.google.com/docs/auth

Google. (s.f.-b). *Firestore*. Firebase. Recuperado el 8 de noviembre de 2026, de https://firebase.google.com/docs/firestore

Google. (s.f.-c). *Firebase documentation*. Firebase. Recuperado el 8 de noviembre de 2026, de https://firebase.google.com/docs

Google. (s.f.-d). *Get realtime updates with Cloud Firestore*. Firebase. Recuperado el 8 de noviembre de 2026, de https://firebase.google.com/docs/firestore/query-data/listen

Google. (s.f.-e). *Get started with Cloud Firestore Security Rules*. Firebase. Recuperado el 8 de noviembre de 2026, de https://firebase.google.com/docs/firestore/security/get-started

Google. (s.f.-f). *Transactions and batched writes*. Firebase. Recuperado el 8 de noviembre de 2026, de https://firebase.google.com/docs/firestore/manage-data/transactions

Moroney, L. (2017). *The definitive guide to Firebase: Build Android apps on Google's mobile platform*. Apress. https://doi.org/10.1007/978-1-4842-2943-9

React Native Async Storage. (s.f.). *Async Storage*. Recuperado el 8 de noviembre de 2026, de https://react-native-async-storage.github.io/async-storage/